Projektoversigt - PingPong Client/Server

Samlet teknisk forklaring af arkitektur, netværk, samtalelogik og konfiguration

1. Formål

Projektet er en mindre **TCP-baseret klient/server-applikation** udviklet i C# og .NET 9. Serveren leverer interaktive "Knock Knock"-jokes, mens klienten viser serverens beskeder i en konsol og sender brugerens svar tilbage.

Løsningen demonstrerer blandt andet:

- kommunikation over TCP,
- en linjebaseret tekstprotokol,
- asynkron behandling af flere klienter,
- dependency injection og options-mønstret,
- validering af konfiguration,
- udskiftelige strategier til valg af jokes,
- en simpel state machine til samtaleforløbet,
- kontrolleret opstart og nedlukning af en server.

2. Teknologier og afgrænsning

Område	Teknologi	Anvendelse
Platform	.NET 9	Kørsel af klient og server
Sprog	C#	Programlogik
Netværk	`TcpListener`, `TcpClient` og `NetworkStream`	TCP-forbindelsen mellem programmerne
Hosting	.NET Generic Host	Serverens levetid og background service
Konfiguration	`appsettings.json` og Options	Port, klientgrænse, strategi og jokes
Validering	Data annotations og `IValidatableObject`	Kontrol af konfiguration ved opstart
Logging	`ILogger<T>`	Information om serverens drift og afbrudte klienter

Projektet bruger ikke HTTP, web-API, database eller grafisk brugerflade. Alle data ligger i serverens konfigurationsfil og i hukommelsen, mens programmet kører.

3. Løsningsstruktur

`PingPongServer.slnx` samler to konsolprojekter, som fysisk ligger i hver sin mappe:

```
PingPongServer.slnx
```

```
|
|-- PingPongServer
|   |-- Domain
|   |-- Networking
|   |-- Options
|   |-- Selection
|   |-- Program.cs
|   `-- appsettings.json
|
`-- ../PingPongClient/PingPongClient
    `-- Program.cs
```

PingPongServer

Serverprojektet har ansvar for:

- at lytte efter TCP-forbindelser,
- at begrænse antallet af samtidige klienter,
- at gennemføre en separat jokesamtale med hver klient,
- at vælge jokes efter den konfigurerede strategi,
- at indlæse og validere konfiguration,
- at logge relevante driftshændelser.

PingPongClient

Klientprojektet har ansvar for:

- at forbinde til serveren på `localhost:5000`,
- at læse beskeder fra netværket,
- at vise beskederne i konsollen,
- at validere, at brugerens input ikke er tomt,
- at sende brugerens svar tilbage som UTF-8-tekst.

4. Overordnet arkitektur

Programmet er opdelt efter ansvar:

```
PingPongClient
      |
      | TCP og UTF-8 tekstlinjer
      v
KnockKnockServer
      |
      v
NetworkConversationSession
      |
      v
KnockKnockConversation
      |
      v
IJokeSelector
      |
      +-- RandomJokeSelector
      `-- SequentialJokeSelector
```

Netværkslaget ved, hvordan tekst læses og skrives over TCP. Domænelaget ved, hvordan jokesamtalen skal forløbe. Selection-laget ved, hvordan den næste joke vælges. Denne opdeling betyder, at samtalelogikken ikke behøver arbejde direkte med `NetworkStream`.

5. Serverens opstart og dependency injection

Program

`Program.Main` opretter en .NET Generic Host og udfører følgende trin:

1. Indlæser `appsettings.json` fra programmets outputmappe.
2. Binder sektionerne til typed options-klasser.
3. Validerer options ved opstart.
4. Registrerer begge jokevælgere i dependency injection-containeren.
5. Vælger den aktive `IJokeSelector` ud fra konfigurationen.
6. Registrerer samtalen og TCP-serveren.
7. Bygger og starter hosten.

Hvis konfigurationen er ugyldig, fanges `OptionsValidationException`. Fejlene udskrives samlet, og programmet afsluttes med fejlkode 1.

Levetider

Service	Levetid	Begrundelse
`RandomJokeSelector`	Singleton	Deler katalog og husker seneste joke i den aktuelle async-kontekst
`SequentialJokeSelector`	Singleton	Deler den trådsikre tæller mellem klienter
`IJokeSelector`	Singleton	Henviser til den valgte strategi
`KnockKnockConversation`	Transient	Indeholder selve samtaleforløbet uden vedvarende tilstand
`KnockKnockServer`	Hosted service	Starter og stopper sammen med applikationen

6. Domænemodeller og samtalelogik

KnockKnockJoke

`KnockKnockJoke` beskriver én joke med:

- `Who`: navnet, som serveren svarer efter "Who's there?",
- `Punchline`: jokernes afsluttende pointe.

Properties er markeret som `required`, så begge værdier skal angives, når objektet oprettes.

ConversationState

Enum-værdierne beskriver, hvor langt samtalen er kommet:

Tilstand	Betydning
`WaitingForWho`	Serveren venter på "Who's there?"
`WaitingForWhoName`	Serveren venter på eksempelvis "Boo who?"
`WaitingForNextJoke`	Serveren venter på `next` eller `quit`
`Finished`	Samtalen afsluttes

KnockKnockConversation

`RunAsync` styrer samtalen som en state machine. Metoden vælger først en joke og sender `Knock! Knock!`. Herefter læses én tekstlinje ad gangen, og den aktuelle tilstand bestemmer, hvilket input der accepteres.

Et normalt forløb er:

```
Server: Knock! Knock!
Klient: Who's there?
Server: Boo
Klient: Boo who?
Server: Don't cry, it's just a joke!
Server: Skriv 'next' for næste joke eller 'quit' for at afslutte.
```

Ved `next` vælges en ny joke, og state machine starter forfra. Ved `quit` sendes `Goodbye!`, og samtalen går til `Finished`.

Metoderne `IsWhoQuestion` og `IsWhoAnswer` accepterer små variationer i tegnsætning og store/små bogstaver. Forkert input giver en vejledende besked uden at flytte samtalen til næste tilstand.

7. Netværkslaget

IConversationSession

Interfacet definerer kun to operationer:

- læs en linje,
- skriv en linje.

Samtalelogikken afhænger af dette interface i stedet for direkte af TCP-klasser. Det reducerer koblingen mellem domæne- og netværkslaget.

NetworkConversationSession

Klassen omsætter en `NetworkStream` til linjebaseret kommunikation:

- `StreamReader` læser UTF-8-tekst indtil linjeskift,
- `StreamWriter` sender UTF-8 uden byte order mark,
- `AutoFlush` sørger for, at hver besked straks sendes,
- `leaveOpen` lader den ydre kode styre selve netværksstreamens levetid.

`Dispose` frigiver reader og writer, når klientsessionen slutter.

KnockKnockServer

`KnockKnockServer` arver fra `BackgroundService` og kører som en del af hosten.

ExecuteAsync

Metoden:

1. starter en `TcpListener` på alle lokale netværksinterfaces,
2. venter asynkront på nye klienter,
3. kontrollerer om en klientplads er ledig,
4. afviser klienten med en forklaring, hvis grænsen er nået,
5. starter ellers en separat task til klienten,
6. registrerer tasken, så den kan afventes ved nedlukning.

HandleClientAsync

Metoden ejer den enkelte `TcpClient`, opretter en `NetworkConversationSession` og starter `KnockKnockConversation.RunAsync`. Uventede netværksafbrydelser logges uden at stoppe hele serveren. Klientpladsen frigives altid i `finally`.

StopAsync

Ved nedlukning stoppes listeneren først. Herefter får den normale host-nedlukning lov til at fortsætte, og serveren venter på de registrerede klienttasks, før `SemaphoreSlim` frigives.

8. Samtidige klienter

Serveren kan behandle flere klienter samtidig.

Mekanisme	Formål
`SemaphoreSlim`	Begrænser aktive klienter til `MaxClients`
`ConcurrentDictionary<Task, byte>`	Holder styr på igangværende klienttasks
`ContinueWith`	Fjerner en task fra samlingen, når den er færdig
`CancellationToken`	Stopper ventende netværksoperationer ved nedlukning
`Interlocked.Increment`	Gør sekventiel jokeudvælgelse trådsikker
`AsyncLocal`	Husker seneste tilfældige joke i den aktuelle async-kontekst

Hvis serveren er fuld, sendes en kort besked til den nye klient, hvorefter forbindelsen lukkes. Eksisterende klienter fortsætter uforstyrret.

9. Jokeudvælgelse

IJokeSelector

Interfacet gør valgstrategien udskiftelig. Samtalelogikken kalder kun `SelectJoke` og behøver derfor ikke kende den konkrete algoritme.

RandomJokeSelector

Vælger et tilfældigt indeks med `Random.Shared`. Hvis kataloget indeholder mere end én joke, gentages den seneste joke ikke straks. Sammenligningen bruger både `Who` og `Punchline`, så dubletter med samme indhold behandles som den samme joke.

SequentialJokeSelector

Bruger en fælles tæller og returnerer jokes i den rækkefølge, de står i konfigurationen. Modulo-beregningen starter forfra ved den første joke, når den sidste er brugt. `Interlocked.Increment` beskytter tælleren, når flere klienter spørger samtidig.

10. Konfiguration og validering

`appsettings.json` har tre hovedsektioner:

Sektion	Options-klasse	Indhold
`Server`	`ServerOptions`	Port og maksimalt antal klienter
`JokeSelection`	`JokeSelectionOptions`	`Random` eller `Sequential`
`Jokes`	`JokeCatalogOptions`	Listen over navne og punchlines

ServerOptions

- `Port` skal være mellem 1 og 65535.
- `MaxClients` skal være mellem 1 og 10000.

JokeSelectionOptions

`Strategy` er påkrævet og må kun være `Random` eller `Sequential`. Sammenligningen ved valg af strategi ignorerer store og små bogstaver.

JokeCatalogOptions

Kataloget skal indeholde mindst én joke. `Validate` gennemgår desuden alle elementer og afviser tomme `Who`- eller `Punchline`-værdier.

Valideringen udføres med `ValidateOnStart`, så serveren ikke begynder at lytte med en ugyldig konfiguration.

11. Klientens programflow

Klientens `Main` udfører følgende:

1. Opretter en `TcpClient`.
2. Forbinder synkront til `localhost` på port 5000.
3. Henter forbindelsens `NetworkStream`.
4. Læser serverdata ind i en buffer på 1024 bytes.
5. Fortolker de modtagne bytes som UTF-8.
6. Viser serverens besked.
7. Beder brugeren om et ikke-tomt svar.
8. Sender svaret med et afsluttende linjeskift.
9. Gentager, indtil serveren lukker forbindelsen eller konsolinput slutter.

Linjeskiftet er vigtigt, fordi serverens `StreamReader.ReadLineAsync` venter på en komplet tekstlinje.

Klientfilen indeholder også ældre, udkommenterede øvelseseksempler. De udføres ikke og påvirker derfor ikke det aktive programflow.

12. Kommunikationsprotokol

Protokollen er tekstbaseret og kører over en vedvarende TCP-forbindelse.

Egenskab	Værdi
Transport	TCP
Standardadresse	`localhost`
Standardport	`5000`
Tegnkodning	UTF-8
Beskedafgrænsning	Linjeskift
Samtalemodel	Request/response styret af serverens state machine

TCP bevarer rækkefølgen af bytes, men er ikke i sig selv beskedorienteret. Serverens linjebaserede reader bruger derfor linjeskift som protokolgrænse.

13. Datahåndtering og database

Projektet har **ingen database**.

- Jokes indlæses fra `appsettings.json` ved opstart.
- Serverkonfigurationen holdes i typed options-objekter.
- Samtalens aktuelle joke og tilstand findes kun i hukommelsen.
- Ingen brugeroplysninger, samtaler eller historik gemmes efter nedlukning.

Hvis projektet senere skal gemme historik eller administrere jokes dynamisk, kan et repository- eller datalag tilføjes uden at ændre selve TCP-protokollen.

14. Fejl- og nedlukningshåndtering

Projektet håndterer flere forventede situationer:

Situation	Håndtering
Ugyldig konfiguration	Fejl vises ved opstart, og processen får fejlkode 1
Klienten afbryder uventet	`IOException` logges for den enkelte klient
Serveren stoppes	Listener og ventende operationer afbrydes via cancellation
Maksimum klienter nået	Den nye klient får en besked og afvises
Klienten lukker input	Klientprogrammet afsluttes
Serveren lukker forbindelsen	Klienten viser besked og forlader løkken

15. Sådan køres løsningen

Start først serveren fra løsningsmappen:

```
dotnet run --project PingPongServer/PingPongServer.csproj
```

Start derefter klienten i en anden terminal:

```
dotnet run --project ../PingPongClient/PingPongClient/PingPongClient.csproj
```

Flere klientterminaler kan startes for at afprøve samtidige samtaler og grænsen i `MaxClients`.

16. Vigtige designvalg

- **TCP frem for HTTP:** Passer til en vedvarende, enkel tekstsamtale.
- **Linjer som beskedgrænse:** Gør protokollen nem at forstå og implementere.
- **State machine:** Gør gyldige svar afhængige af samtalens aktuelle trin.
- **Interface omkring sessionen:** Holder domænelogik fri af direkte netværkskode.
- **Strategy pattern:** Gør tilfældig og sekventiel udvælgelse udskiftelig.
- **Options og opstartsvalidering:** Fejl i port, strategi eller jokes opdages før serverstart.
- **SemaphoreSlim:** Giver en enkel grænse for samtidige klienter.
- **Hosted service:** Giver kontrolleret levetid, logging og graceful shutdown.

17. Samlet ansvarskort

Type	Primært ansvar
`Program`	Konfiguration, dependency injection og opstart
`KnockKnockServer`	TCP-listener, klientstyring og nedlukning
`NetworkConversationSession`	Linjebaseret UTF-8-kommunikation
`IConversationSession`	Abstraktion af samtalens input og output
`KnockKnockConversation`	State machine og dialogregler
`ConversationState`	Samtalens aktuelle trin
`KnockKnockJoke`	Data for én joke
`IJokeSelector`	Fælles kontrakt for jokevalg
`RandomJokeSelector`	Tilfældigt valg uden øjeblikkelig gentagelse
`SequentialJokeSelector`	Trådsikkert valg i fast rækkefølge
Options-klasserne	Typed og valideret konfiguration
Klientens `Program`	Konsolinput og TCP-kommunikation med serveren